CORE103L Computational Thinking
LW10 - Recursion I



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions 2
1.2 Marking scheme 2
1.2.1 Lab Objectives 2
1.3 Late submission policy 2
1.4 Use of AI 3
1.5 Viva 3

2 Prerequisites **4**
2.1 Assert Statements 4
2.2 Pytest 4

3 Lab Exercises **6**
3.1 Deadline Countdown 6
3.2 Guess My Midterm Score 8
3.3 String Search 10

Introduction

1.1 Instructions

- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW10_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

Following are the lab objectives of this lab:

- Understand the fundamental concept of recursion in programming.
- Differentiate between recursive functions and iterative solutions.
- Explain the structure of recursive algorithms, including base cases and recursive calls.
- Write simple recursive functions to solve basic problems.

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
    assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
    pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
    pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
    pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================== 10 passed in 0.06s =========
```

However, if all the test fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================= test session starts =============================
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFFFF

================================== FAILURES ===================================
_____________________________ test_question1[lst10-lst20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ___
```

Lab Exercises

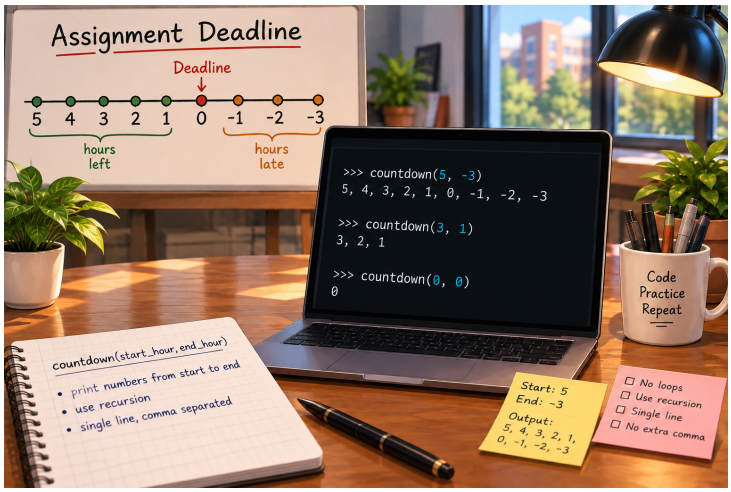
3.1 Deadline Countdown

This question should be implemented in the file ***q1.py****.*

Problem Description

Every freshman knows the feeling of an assignment deadline getting closer. The course portal sends a reminder every hour near the deadline, starting some hours before it and continuing through the late-submission window. Each reminder carries a number: positive numbers are the hours left, **0** is the deadline itself, and negative numbers are the hours since the deadline, when the submission is late.

Before an assignment opens, the TA wants to preview the reminder schedule.



Function Description

Write a **recursive** function **countdown** that takes as parameters start_hour and end_hour and prints the number of every reminder, from start_hour down to end_hour (inclusive), on a **single line**, separated by a comma and a space, with nothing after the last number.

The function prints and returns nothing.

Sample Interaction

```
>>> countdown(5, -3)
5, 4, 3, 2, 1, 0, -1, -2, -3

>>> countdown(3, 1)
3, 2, 1

>>> countdown(0, 0)
0
```

Sample Interaction Explanation

Call	Meaning
`countdown(5, -3)`	reminders from 5 hours before the deadline to 3 hours after it
`countdown(3, 1)`	reminders from 3 hours before to 1 hour before the deadline
`countdown(0, 0)`	a single reminder, at the deadline itself

When start_hour and end_hour are the same number, only that number is printed, with no comma.

Constraints

- start_hour and end_hour are integers, and $-24 \leq$ end_hour $\leq$ start_hour ≤ 72.
- You must use **recursion**. You may not use loops or any other form of iteration, and you may not use global variables.
- You are not allowed to create a helper function.

Testing

In order to test your function, type the following command on the terminal:

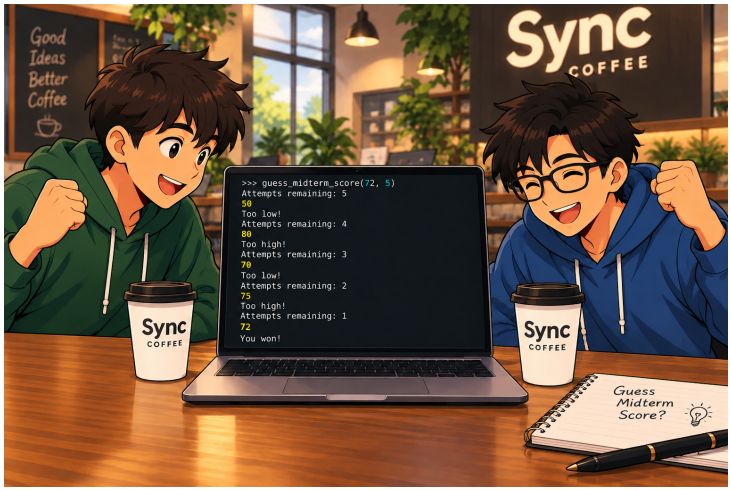
```
pytest tests/test_q1.py
```

3.2 Guess My Midterm Score

This question should be implemented in the file ***q3.py****.*

Problem Description

Midterm results are out, and your friend refuses to tell you their score. Instead, they challenge you: guess it within a limited number of attempts and they buy you your favorite Sync coffee, otherwise you buy theirs. After each wrong guess, they only say whether it was too high or too low.



Function Description

Write a **recursive** function **guess_midterm_score** that takes as parameters midterm_score and attempts_left that simulates the game. Here, midterm_score is your friend's secret score. Before each guess, the function prints the attempts remaining, then reads your guess with **input()**. The game ends when you guess the score or run out of attempts. Everything the program prints is your friend speaking.

Messages

Print exactly these lines (`<...>` marks a value):

```
Attempts remaining: <attempts_left>
Too low!
Too high!
You won with <attempts_after> attempts remaining.
You lost!
```

- Before every guess, print `Attempts remaining: ...`.
- If the guess is correct, print the win message and stop. `<attempts_after>` is the number of attempts left *after* the winning guess.
- If the guess is wrong and attempts remain, print `Too low!` or `Too high!` and continue.
- If the guess is wrong and it was the last attempt, print **only** `You lost!` (no hint).

Constraints

- You must use **recursion**. You may not use loops or any other form of iteration, and you may not use global variables.
- You are not allowed to create a helper function.
- The function prints and returns nothing.
- **input()** returns a string. Each guess is an integer, so convert it.
- The midterm is out of 100, so $0 \leq$ midterm_score ≤ 100.
- $1 \leq$ attempts_left ≤ 19.
- Each guess is assumed to be a valid integer. Input validation is not required.

Sample Interaction

In the samples, `[ Input -> x ]` shows what you type. Your program does not print it.

```
>>> guess_midterm_score(72, 5)
Attempts remaining: 5
[ Input -> 50 ]
Too low!
Attempts remaining: 4
[ Input -> 75 ]
Too high!
Attempts remaining: 3
[ Input -> 60 ]
Too low!
Attempts remaining: 2
[ Input -> 70 ]
Too low!
Attempts remaining: 1
[ Input -> 65 ]
You lost!

>>> guess_midterm_score(85, 10)
Attempts remaining: 10
[ Input -> 50 ]
Too low!
Attempts remaining: 9
[ Input -> 75 ]
Too low!
Attempts remaining: 8
[ Input -> 85 ]
You won with 7 attempts remaining.

>>> guess_midterm_score(100, 1)
Attempts remaining: 1
[ Input -> 100 ]
You won with 0 attempts remaining.
```

Sample Interaction Explanation

In the first sample, your friend scored 72 and you have 5 attempts:

Attempts remaining	Your guess	Your friend says
5	50	Too low!
4	75	Too high!
3	60	Too low!
2	70	Too low!
1	65	You lost! (wrong on the last attempt)

In the second sample, your friend scored 85 and you have 10 attempts. You guess correctly on the third try, so $10 - 3 = 7$ attempts remain.

In the third sample, your friend scored a perfect 100, and your only attempt is correct, so you win with 0 attempts remaining.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 String Search

This question should be implemented in the file ***q5.py****.*

Problem Description

When you press Ctrl+F in your lecture notes, the computer finds the first place where your keyword appears. In this question, you will build that search yourself.

Positions are numbered from **0**, so the first character of the notes is at position **0**.



Function Description

Write a **recursive** function **my_find** that takes two strings, notes and keyword. It must **print** the position of the first occurrence of keyword in notes, searching from the left. If keyword does not appear in notes, it must print **-1**. The function returns nothing.

You must also define a recursive helper function that is called by **my_find**. You may choose its name and parameters.

Sample Interaction

```
>>> my_find('banana', 'na')
2

>>> my_find('ban', 'na')
-1

>>> my_find('banana', 'ban')
0

>>> my_find('aaaa', 'aa')
0

>>> my_find('', 'a')
-1
```

Sample Interaction Explanation

In the first example, we check each starting position in **'banana'** for a match with **'na'**:

Position	Next 2 letters	Match?
0	`ba`	no
1	`an`	no
2	`na`	**yes, print 2**

In the second example, **'ban'** has no two characters starting at position **2**, so the search stops without a match and prints **-1**.

In the fourth example, the first match is at position **0**, so the overlapping match at position **1** is not considered.

In the final example, the notes are empty, so the function prints **-1**.

Hint

Your helper can take an extra parameter that keeps track of the position currently being checked.

Constraints

- $0 \leq$ **len(notes)** ≤ 500.
- $1 \leq$ **len(keyword)** ≤ 500.
- You must use **recursion** through a helper function called by **my_find**.
- You may not use loops, comprehensions, or any other form of iteration.
- Matching is case-sensitive.
- You may not use global variables.
- You may not use string methods or the **in** operator. You may use **len()**, indexing, slicing, equality, integer arithmetic, and comparisons.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```